

Honeypot redirection in ambiente SDN

Luca Esposito DE900074, Luca Pesacane DE900199

1 Introduzione

L'obiettivo del project work è quello di realizzare, all'interno di un ambiente **SDN**, il **rilevamento** del traffico di un potenziale attaccante e la **redirezione** di quest'ultimo verso un **honeypot**, ossia una finta copia del server, senza che l'hacker se ne accorga.

Tale tecnica è volta non solo a mitigare gli effetti di un attacco indirizzato al server esposto, ma di spostarlo in un contesto isolato laddove possa essere osservato, così da **raccogliere informazioni** utili in chiave **CTI**, come tecniche impiegate e servizi contattati.

Affinché ciò sia possibile bisogna, quindi, mettere a punto dei meccanismi che consentano di nascondere la redirezione dell'hacker, così da non fargli notificare il fatto di essere stato scoperto e, quindi, continuare ad osservare il pattern di attacco messo in pratica.

In particolare il progetto tratta il rilevamento di due tipologie differenti di attacco a partire dalle statistiche ottenute dal **protocollo OpenFlow**, l'implementazione della redirezione trasparente mediante riscrittura degli header e la loro successiva verifica su una topologia emulata in **Mininet**.

2 Topologia

La rete emulata è composta da due **switch OpenFlow** tra loro collegati e un **controller Ryu**, connesso in remoto ad entrambi, che li programma mediante OpenFlow.

Sono presenti inoltre sei host: un server, un honeypot, un host attaccante e tre benevoli, di cui due contattano i servizi esposti sulle porte TCP e uno sulla porta UDP.

La topologia è stata realizzata mediante uno script Python che ne definisce la struttura, mentre la rappresentazione grafica è stata ottenuta con il tool grafico MiniEdit.

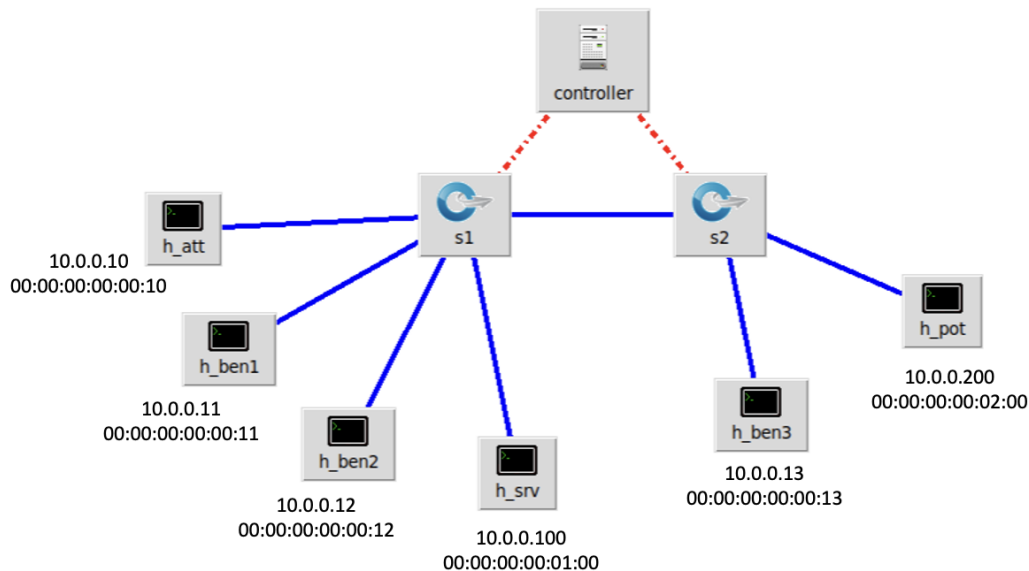


Figura 1: Topologia in MiniEdit

2.1 Server / Honeypot

Espongono lo stesso insieme di servizi TCP e un servizio UDP, avviati con `iperf`, utilizzato per simulare la presenza di servizi in ascolto su più porte:

```
for p in 5001 5002 5003 5004 5005; do iperf -s -p $p & done
iperf -s -u -p 5010 &
wait
```

L'honeypot ha esattamente lo stesso script del server così da replicare i servizi della vittima affinché l'attaccante, una volta dirottato, ritrovi le porte attese anziché trovarle chiuse.

2.2 Client

Due di essi generano traffico TCP legittimo verso il server, con sessioni brevi su porte scelte casualmente e attesa distribuita esponenzialmente fra una sessione e l'altra, in modo da riprodurre un processo di Poisson, tipicamente usato per modellare utenti indipendenti:

```
while true; do
    iperf -c 10.0.0.100 -p 500$((RANDOM%5+1)) -n 200K
    sleep $(awk -v s=$RANDOM 'BEGIN{srand(s); print -log(1-rand())/2}')
done
```

Un ulteriore host genera un flusso UDP costante, per verificare che anche un pattern di traffico continuo non venga classificato erroneamente come sospetto:

```
while true; do iperf -c 10.0.0.100 -u -p 5010 -b 64k -t 60; done
```

In generale forniscono una baseline rispetto alla quale tarare i meccanismi di riconoscimento degli attacchi e rappresentano il traffico che va instradato normalmente senza generare allarmi.

2.3 Hacker

Genera il traffico malevolo indirizzato al server reale, simulando un attacco verso quest'ultimo, e viene impiegato per verificare che la logica di riconoscimento e dirottamento funzionino correttamente.

I pattern di attacco eseguiti dall'hacker sono descritti nella sezione ad essi dedicata.

3 Isolamento dell'honeypot

Un honeypot è pensato per essere raggiunto e, quindi, potenzialmente compromesso da un attaccante, pertanto deve essere opportunamente isolato dal resto della rete, altrimenti potrebbe essere sfruttato dall'hacker per i suoi fini.

Per realizzare l'isolamento, quindi, vengono installate delle **regole statiche** non appena l'honeypot si connette al controller e prima di iniziare a trasportare traffico di rete.

In particolare, tutto il traffico originato dall'honeypot viene scartato, salvo le **regole dinamiche** di eccezione installate dal controller verso gli host dirottati al momento della loro rilevazione.

Questa distinzione è dovuta al fatto che l'isolamento non dipende dall'osservazione di un comportamento, ma è una proprietà che deve valere fin dal primo istante di funzionamento della rete.

Non appena si avviano controller e topologia è possibile verificare le regole installate nello switch s2 con seguente comando:

```
ovs-ofctl -O OpenFlow13 dump-flows s2
```

```
cookie=0x0, duration=1791.368s, table=0, n_packets=0, n_bytes=0, priority=50,ip,nw_src=10.0.0.200 actions=drop
cookie=0x0, duration=1791.368s, table=0, n_packets=3, n_bytes=126, priority=60,arp,dl_src=00:00:00:00:02:00 actions=CONTROLLER:65535
```

Figura 2: Flow Entry di isolamento sullo switch 2

La prima regola consente all'honeypot di emettere richieste ARP, necessarie a risolvere l'indirizzo di un host verso cui deve rispondere, utile quando deve fingersi server nell'interazione con l'hacker, mentre la seconda scarta ogni pacchetto IP la cui sorgente sia l'honeypot.

Per verificare l'efficacia delle regole di isolamento è stato effettuato un test utilizzando il comando pingall da mininet:

```
mininet> pingall
*** Ping: testing ping reachability
h_att -> h_ben1 h_ben2 h_srv h_ben3 X
h_ben1 -> h_att h_ben2 h_srv h_ben3 X
h_ben2 -> h_att h_ben1 h_srv h_ben3 X
h_srv -> h_att h_ben1 h_ben2 h_ben3 X
h_ben3 -> h_att h_ben1 h_ben2 h_srv X
h_pot -> X X X X X
*** Results: 33% dropped (20/30 received)
```

Figura 3: Esito pingall

Dall'immagine si può notare come l'honeypot non sia raggiungibile da alcun host nella rete e che quest'ultimo non sia in grado di raggiungere a sua volta nessun host.

4 Attacchi

Sono state considerate due tipologie di attacco, selezionate per la loro capacità di rappresentare due minacce fondamentali nell'ambito della sicurezza delle reti. La prima punta alla **conoscenza della rete** senza alterarne il servizio, mentre la seconda punta all'**interruzione del servizio** tramite la saturazione delle risorse.

I due tipi di attacchi quindi sono molto diversi e necessitano un'implementazione di criteri di detection diversi da parte del controller per ciascuno di essi.

4.1 Port scanning

Ha l'obiettivo di raccogliere informazioni sul bersaglio, quali porte aperte o servizi attivi, senza comprometterne necessariamente la disponibilità, inviando una serie di scansioni rapide verso l'host bersaglio. Per realizzare questo comportamento è stato utilizzato lo strumento nmap nel terminale mininet dell'host hacker:

```
nmap -sS -p 1-6000 10.0.0.100
```

L'informazione rilevante che permette di distinguere questo tipo di attacco dal traffico legittimo non risiede nel volume complessivo ma nell'**elevato numero di porte distinte raggiunte in un breve intervallo di tempo**.

4.2 Flood volumetrico

Ha l'obiettivo di esaurire le risorse del bersaglio inondandolo di richieste, al fine di rendere i servizi esposti non raggiungibili. È stato realizzato in due varianti, UDP e TCP, dirette verso una singola porta:

```
iperf -c 10.0.0.100 -u -p 5010 -b 3M -t 40
```

```
iperf -c 10.0.0.100 -p 5001 -t 40
```

Si presenta con un profilo opposto rispetto alla tipologia di attacco precedente, cioè come un enorme volume di traffico concentrato verso poche destinazioni o un unico host, già noto dal primo pacchetto. In questo caso, quindi, l'informazione rilevante risiede nell'**elevato rate di trasmissione** e non nella cardinalità delle porte contattate.

4.2.1 TCP VS UDP

Il flooding è stato eseguito nelle due varianti che hanno evidenziato comportamenti diversi a causa della natura dei protocolli di trasporto impiegati:

- Nel caso di flood UDP (**stateless**) ogni datagramma è indipendente, pertanto l'honeypot lo accetta senza bisogno di alcun contesto pregresso, e la redirection risulta trasparente fin dal primo pacchetto deviato.
- Nel caso del flood TCP (**stateful**), la connessione stabilita inizialmente con il server reale prima della rilevazione viene interrotta da un RST dell'honeypot, poichè OpenFlow non può riscrivere i numeri di sequenza e l'honeypot non può continuare la sessione precedente. In questo caso il RST giunge all'attaccante dall'indirizzo atteso, risultando indistinguibile da una chiusura del servizio. Le connessioni instaurate dopo la mitigazione, corrispondenti a tentativi di attacco successivi, vengono invece dirottate senza alcuna interruzione.

5 Detection

A causa della diversa natura delle tipologie di attacco simulate, è stato necessario considerare due canali di raccolta dati specifici di OpenFlow per poterne eseguire il riconoscimento:

- **PacketIn**: funge da canale **asincrono**, basato sulle richieste in arrivo dagli switch in seguito ai **table-miss**: ogni pacchetto che non trova corrispondenza nella flow table risale al controller. È immediato, ma vede esclusivamente il primo pacchetto di ogni flusso, per cui è adatto a rilevare la cardinalità delle destinazioni tipica del port scan.
- **FlowStats**: rappresenta il canale **sincrono** in quanto è alimentato dal **polling**: il controller interroga gli switch ogni 2 secondi e riceve, per ogni flow entry, i contatori **packet_count** e **byte_count**, ovvero quanti pacchetti/byte sono transitati su ciascuna regola già installata. È l'unico canale in grado di rendere visibile il volume di un flood, poichè una connessione unica genera un solo evento asincrono iniziale e resta poi interamente nel data plane.

Sulla base delle statistiche raccolte durante la generazione del traffico dei client benevoli sono state scelte delle soglie da usare per il confronto, riportate nella seguente tabella:

	Port scan	Flood volumetrico
Fonte di informazione	PacketIn (asincrono)	FlowStats (polling)
Grandezza discriminante	porte distinte in 5 s	pacchetti al secondo
Condizione	$n_{\text{ports}} \geq 20 \wedge n_{\text{flows}} \geq 20$	$pps \geq 500 \wedge n_{\text{ports}} \leq 5$

Tabella 1: Criteri di classificazione dei due attacchi

Le soglie sono state calibrate sul traffico generato con **iperf**, il cui carattere è più regolare rispetto a quello di una rete reale, e andrebbero quindi ricalibrate in un contesto operativo.

6 Redirection

A differenza dei meccanismi di detection, che differiscono a seconda della natura dell'attacco, il meccanismo di redirection è indipendente dal tipo di attacco che lo ha innescato.

L'obiettivo è quello di operare sui pacchetti in modo da indirizzare quelli che hanno come mittente l'ip precedentemente identificato come malevolo verso l'honeypot, modificando le informazioni contenute nell'header mediante l'applicazione di regole OpenFlow.

In particolare, la **risrittura degli indirizzi** avviene interamente sullo switch s1, mentre sullo switch s2, collegato all'honeypot, viene invece installata la sola **eccezione dinamica** all'isolamento, che consente l'uscita del pacchetto di risposta senza applicare alcuna modifica agli indirizzi.

Al fine di verificare l'applicazione delle regole all'interno degli switch sono stati utilizzati i seguenti comandi nella CLI mininet:

```
ovs-ofctl -O OpenFlow13 dump-flows s1
```

```
cookie=0xbeef, duration=18.903s, table=0, n_packets=1917, n_bytes=111186, idle_timeout=60, send_flow_rem priority=100, ip,nw_src=10.0.0.10,nw_dst=10.0.0.100 actions=set_field:10.0.0.200->ip_dst,set_field:00:00:00:00:02:00->eth_dst,output:5
cookie=0xbeef, duration=18.903s, table=0, n_packets=1917, n_bytes=103518, idle_timeout=60, priority=100, ip,nw_src=10.0.0.200,nw_dst=10.0.0.10 actions=set_field:10.0.0.100->ip_src,set_field:00:00:00:00:01:00->eth_src,output:1
```

Nello **switch s1** sono presenti due regole che realizzano la riscrittura bidirezionale degli indirizzi:

- La prima intercetta i pacchetti provenienti dall'attaccante e diretti al server target, sostituendo l'indirizzo IP e MAC di destinazione con quelli dell'honeypot.
- La seconda opera in senso inverso, intercettando i pacchetti di risposta provenienti dall'honeypot e diretti all'attaccante, sostituendo l'indirizzo IP e MAC di sorgente con quelli del server vero, in modo tale che l'attaccante continui a credere di comunicare con quest'ultimo.

```
ovs-ofctl -O OpenFlow13 dump-flows s2
```

```
cookie=0xbeef, duration=12.002s, table=0, n_packets=1888, n_bytes=101952, idle_timeout=60, priority=60, ip,nw_src=10.0.0.200,nw_dst=10.0.0.10 actions=CONTROLLER:65535
```

Nello **switch s2**, a cui l'honeypot è collegato, è presente solo la regola di eccezione dinamica che consente ai pacchetti di risposta, aventi come sorgente l'honeypot e come destinazione l'attaccante, di essere inoltrati verso **s1** anziché scartati dalla regola di isolamento descritta in precedenza.

Gli effetti della redirectione sono stati osservati anche mediante cattura **Wireshark**, in cui sono stati analizzati i flussi di pacchetti dell'host attaccante e dell'honeypot.

No.	Time	Source	Destination	Protocol	Length	Info
2728	20.312667150	10.0.0.200	10.0.0.10	TCP	54	1718 → 44953 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
2729	20.326574023	10.0.0.10	10.0.0.200	TCP	58	44953 → 374 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
2730	20.327788675	10.0.0.200	10.0.0.10	TCP	54	374 → 44953 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
2731	20.332307981	10.0.0.10	10.0.0.200	TCP	58	44953 → 323 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
2732	20.333363739	10.0.0.200	10.0.0.10	TCP	54	323 → 44953 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0

Figura 4: Cattura Wireshark hacker

Sull'interfaccia dell'attaccante si può notare come, anche dopo la redirectione, i pacchetti continuino ad essere trasmessi e soprattutto ricevuti avendo come indirizzo quello del server reale, così da non consentirgli di accorgersi del fatto che in realtà i pacchetti stiano arrivando all'honeypot.

No.	Time	Source	Destination	Protocol	Length	Info
4969	20.446511896	10.0.0.10	10.0.0.100	TCP	58	44954 → 323 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
4970	20.503793925	10.0.0.10	10.0.0.100	TCP	58	44954 → 1146 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
4971	20.915366913	10.0.0.100	10.0.0.10	TCP	54	374 → 44953 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
4972	20.915367513	10.0.0.100	10.0.0.10	TCP	54	323 → 44953 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
4973	20.915472671	10.0.0.100	10.0.0.10	TCP	54	1146 → 44953 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0

Figura 5: Cattura Wireshark honeypot

A conferma di quanto detto, nell'interfaccia dell'honeypot si può notare come gli stessi pacchetti arrivino a quest'ultimo grazie alla riscrittura degli indirizzi operata dallo switch.

Al termine della comunicazione, dalla CLI Mininet si è testato il comando ARP per verificare che effettivamente l'host attacker, nonostante la redirectione subita, possieda all'interno della propria cache solamente l'indirizzo MAC del server reale.

```
h_att arp -n
```

```
Address      HWtype  HWaddress      Flags Mask    Iface
10.0.0.100   ether   00:00:00:00:01:00  C             h_att-eth0
```